

LAB 1: Linear Regression

Giới thiệu

Hồi quy tuyến tính (Linear Regression) là một trong những thuật toán cơ bản trong Machine Learning, đóng vai trò như một điểm khởi đầu để hiểu các nguyên tắc cơ bản của predictive modeling. Trong bài thực hành này, bạn sẽ triển khai một mô hình Linear Regression từ đầu và áp dụng nó vào dữ liệu thực tế.

Mục tiêu

- Hiểu nguyên lý cơ bản của bài toán hồi quy tuyến tính, tối ưu hàm mất mát và phương pháp gradient descent.
- Triển khai mô hình Linear Regression chi tiết từng bước, áp dụng mô hình vào dữ liệu thực tế và đánh giá hiệu suất.
- Tìm hiểu về hiện tượng overfitting và bias-variance tradeoff trong mô hình hồi quy.

Phần I: Tối ưu Gradient Descent

Bài 1

Thực hiện 3 bước Gradient Descent với learning rate $\alpha = 0.1$ để tìm cực tiểu của các hàm số sau, bắt đầu từ điểm khởi tạo đã cho:

- (a) $f(x) = x^2 - 4x + 4$ với điểm khởi tạo $x_0 = 0$.
- (b) $f(x) = x^4 - 3x^3 + 2$ với điểm khởi tạo $x_0 = 1$.
- (c) $f(x) = \sin(x) + \frac{x^2}{10}$ với điểm khởi tạo $x_0 = 2$.
- (d) $f(x) = e^{0.5x} - 4x$ với điểm khởi tạo $x_0 = 0$.

Bài 2

Cho hàm số $f(x) = x^2 - 6x + 5$ với điểm khởi tạo $x_0 = 0$. Thực hiện 5 bước Gradient Descent với các learning rate khác nhau: (i) $\alpha = 0.01$, (ii) $\alpha = 0.1$, (iii) $\alpha = 0.5$, (iv) $\alpha = 1.0$. Với mỗi learning rate:

- (a) Tính toán giá trị x sau mỗi bước lặp.
- (b) Vẽ biểu đồ thể hiện sự thay đổi của $f(x)$ theo số bước lặp.
- (c) Nhận xét về tốc độ hội tụ và độ ổn định của thuật toán.
- (d) Xác định learning rate tối ưu từ các thử nghiệm trên.

Bài 3

Cho hàm số $f(x, y) = (x - 2)^2 + (y + 1)^2 + 3$ với điểm khởi tạo $(x_0, y_0) = (0, 0)$ và learning rate $\alpha = 0.1$.

Thực hiện 5 bước Gradient Descent:

- (a) Tính đạo hàm riêng: $\frac{\partial f}{\partial x}$ và $\frac{\partial f}{\partial y}$.
- (b) Tính toán giá trị (x, y) sau mỗi bước lặp sử dụng quy tắc cập nhật:

$$\begin{aligned}x_{t+1} &:= x_t - \alpha \frac{\partial f}{\partial x} \\y_{t+1} &:= y_t - \alpha \frac{\partial f}{\partial y}\end{aligned}$$

- (c) Ghi lại giá trị hàm số $f(x_t, y_t)$ và gradient norm $\|\nabla f\| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$ ở mỗi bước.
- (d) Nhận xét về sự hội tụ của thuật toán và so sánh với các bài toán một biến ở trên.

Phần II: Mô hình hồi quy tuyến tính

Bài 1: Xây dựng mô hình hồi quy đơn biến

Cho tập dữ liệu về mối quan hệ giữa số giờ học tập (X) và điểm thi (y):

Giờ học (X)	Điểm thi (y)
1	2
2	4
3	5
4	4
5	5

Yêu cầu:

- (a) Tính toán các tham số w_0 (bias) và w_1 (weight) của mô hình $\hat{y} = w_0 + w_1 \cdot x$ bằng phương pháp bình phương tối thiểu (Ordinary Least Squares - OLS):

$$w_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$w_0 = \bar{y} - w_1 \bar{x}$$

Trong đó $\bar{x}$ và $\bar{y}$ là giá trị trung bình của X và y .

- (b) Viết phương trình đường hồi quy đơn biến.
- (c) Dự đoán điểm thi khi học 6 giờ.
- (d) Tính Mean Squared Error (MSE) và R-squared (R^2) của mô hình trên tập dữ liệu này.

Bài 2: Xây dựng mô hình hồi quy đa biến

Cho tập dữ liệu về giá nhà với hai đặc trưng đầu vào: diện tích (x_1) và số phòng ngủ (x_2):

Diện tích (x_1)	Số phòng (x_2)	Giá nhà (y)
1000	2	200
1500	3	300
2000	4	400
1200	2	220
1800	3	350

Mô hình hồi quy đa biến có dạng:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2$$

Yêu cầu:

- (a) Chuẩn bị dữ liệu: Xây dựng ma trận thiết kế $\mathbf{X}$ và vector mục tiêu $\mathbf{y}$, gồm:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & x_{2,1} \\ 1 & x_{1,2} & x_{2,2} \\ \vdots & \vdots & \vdots \\ 1 & x_{1,n} & x_{2,n} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

- (b) Tính toán các tham số $\mathbf{w} = (w_0, w_1, w_2)^T$ bằng công thức Normal Equation:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- (c) Viết phương trình mô hình hồi quy đa biến sau khi tìm được các tham số.
- (d) Dự đoán giá nhà có diện tích 1600 m^2 và 3 phòng ngủ.
- (e) Tính MSE và R^2 của mô hình trên tập dữ liệu này.
- (f) So sánh hiệu suất mô hình này với mô hình đơn biến (Bài 1) - giải thích ưu điểm của việc sử dụng nhiều đặc trưng.

Bài 3: Dự đoán doanh số bán hàng dựa trên chi phí quảng cáo

Trong bài tập này, các bạn sẽ thực hành triển khai mô hình hồi quy tuyến tính với tập dữ liệu Advertising (đính kèm trên classroom). Tập dữ liệu này chứa thông tin về chi phí quảng cáo trên các kênh truyền thông và doanh số bán hàng tương ứng của các công ty.

Yêu cầu 1: Chuẩn bị và khảo sát dữ liệu

- Tải file *advertising.csv* và sử dụng thư viện Pandas để đọc file.
- Trả lời các câu hỏi sau:
 - Tập dữ liệu có bao nhiêu dòng và bao nhiêu cột?
 - Các trường dữ liệu cho biết những thông tin gì?
 - Tập dữ liệu có tồn tại các giá trị thiếu (NaN hay Null) không?
 - Mối quan hệ giữa các đặc trưng với biến mục tiêu như thế nào? Hãy sử dụng biểu đồ scatter plot để trực quan hóa mối quan hệ này.
 - Nhận xét sơ bộ về phân phối của các đặc trưng và biến mục tiêu.

Yêu cầu 2: Tiền xử lý dữ liệu

- Kiểm tra và xử lý các giá trị thiếu (nếu có).
- Trích xuất đặc trưng *TV* làm biến đầu vào (X) và *Sales* làm biến mục tiêu (y).
- Chia tập dữ liệu thành tập huấn luyện (X_{train}, y_{train}) và tập kiểm tra (X_{test}, y_{test}) với tỷ lệ 80:20.
- Chuẩn hóa các đặc trưng sử dụng một trong hai phương pháp sau:
 - Min-Max Scaling: Đưa dữ liệu về khoảng giá trị $[0, 1]$.

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- Standardization: Chuyển đổi giá trị về phân phối chuẩn với trung bình 0 và độ lệch chuẩn 1.

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

Note:

Thông thường, tập dữ liệu sẽ được chia thành 2 phần (tập huấn luyện và tập kiểm tra). Tùy thuộc vào kích thước của tập dữ liệu, bạn có thể điều chỉnh tỉ lệ này, ví dụ 70-30 hay 80-20.

Tập kiểm tra sẽ được sử dụng để đánh giá hiệu suất mô hình sau khi huấn luyện xong, bạn cần đảm bảo tập dữ liệu này không bị rò rỉ thông tin trong quá trình tiền xử lý và huấn luyện mô hình. Nếu cần thiết, bạn có thể sử dụng thêm tập xác thực (validation set) để điều chỉnh siêu tham số của mô hình. Tập xác thực sẽ được tách ra từ tập huấn luyện trước khi huấn luyện mô hình.

Chuẩn hóa dữ liệu giúp mô hình học hiệu quả hơn bằng cách đưa các đặc trưng về cùng thang đo. Việc này giúp cải thiện quá trình tối ưu hóa và đảm bảo mô hình phản ánh chính xác mối quan hệ giữa đặc trưng và biến mục tiêu. Thực hiện chuẩn hóa dữ liệu là đặc biệt quan trọng với các thuật toán nhạy cảm với phạm vi giá trị như Gradient Descent.

Yêu cầu 3: Xây dựng và huấn luyện mô hình hồi quy tuyến tính đơn biến

Từ đặc trưng đã chọn ở yêu cầu 2, mô hình hồi quy tuyến tính sẽ có dạng:

$$\hat{Sales} = w_0 + w_1 \cdot TV$$

Trong đó:

- $\hat{Sales}$ là giá trị doanh số bán hàng dự đoán.
- w_0 là trọng số bias (intercept).
- w_1 là trọng số (weight) tương ứng với đặc trưng TV.

Hoàn thiện hàm `predict()` để dự đoán giá trị doanh số bán hàng dựa trên chi phí quảng cáo trên TV:

```

1 def predict(X, w0, w1):
2     """
3     Params:
4         X: np.array, shape (n_samples, ) - input feature (TV
           advertising cost)
5         w0: float - bias weights
6         w1: float - TV weight
7     Returns:
8         y_pred: np.array, shape (n_samples, ) - predicted sales
9     """
10    pass

```

Hàm mất mát (loss function hay cost function) được sử dụng để đánh giá hiệu suất của mô hình hồi quy tuyến tính là Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Trong đó:

- n là số lượng mẫu trong tập dữ liệu.
- y_i là giá trị thực tế của doanh số bán hàng.
- $\hat{y}_i$ là giá trị dự đoán của doanh số bán hàng từ mô hình.

Hoàn thiện hàm `loss_function()` để tính toán MSE giữa giá trị thực tế và giá trị dự đoán:

```

1 def loss_function(y_true, y_pred):
2     """
3     Params:
4         y_true: np.array, shape (n_samples, ) - true sales
5         y_pred: np.array, shape (n_samples, ) - predicted sales
6     Returns:
7         mse: float - Mean Squared Error
8     """
9     pass

```

Việc tính toán gradient của hàm mất mát sẽ cho ta biết hướng và độ lớn của bước cập nhật trọng số trong quá trình huấn luyện mô hình. Với việc sử dụng 1 đặc trưng đầu vào, các đạo hàm riêng của hàm mất mát MSE theo các trọng số w_0 và w_1 được tính như sau:

$$\frac{\partial L}{\partial w_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

$$\frac{\partial L}{\partial w_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot x_i$$

Hoàn thiện hàm `gradient()` để tính toán gradient của hàm mất mát:

```

1 def gradient(X, y, w0, w1):
2     """
3     Params:
4         X: np.array, shape (n_samples, ) - input feature (TV
           advertising cost)
5         y: np.array, shape (n_samples, ) - true sales
6         w0: float - bias weights
7         w1: float - TV weight
8     Returns:
9         dw0: float - gradient w.r.t w0
10        dw1: float - gradient w.r.t w1
11    """
12    pass

```

Tiếp theo, ta sẽ huấn luyện mô hình bằng cách cập nhật các trọng số w_0 và w_1 sử dụng thuật toán Gradient Descent. Quá trình cập nhật trọng số sẽ được diễn ra trong một số vòng lặp nhất định (epochs), với mỗi vòng lặp ta sẽ tính toán gradient và cập nhật trọng số như sau:

$$w_0 := w_0 - \alpha \cdot \frac{\partial L}{\partial w_0}$$

$$w_1 := w_1 - \alpha \cdot \frac{\partial L}{\partial w_1}$$

Tham số α trong công thức là learning rate, quyết định khoảng cách mà trọng số được cập nhật trong mỗi bước. Việc lựa chọn learning rate phù hợp là rất quan trọng để đảm bảo quá trình huấn luyện diễn ra hiệu quả. Hãy sử dụng hàm `fit()` bên dưới để huấn luyện và theo dõi quá trình mô hình học:

```

1 def fit(X, y, learning_rate=0.01, epochs=1000):
2     learning_rate = learning_rate
3     epochs = epochs
4     losses = []
5     w0, w1 = 0.0, 0.0 # Initialize weights
6     for epoch in range(epochs):
7         y_pred = predict(X, w0, w1)
8         dw0, dw1 = gradient(X, y, w0, w1)
9         w0 -= learning_rate * dw0
10        w1 -= learning_rate * dw1
11        loss = loss_function(y, y_pred)
12        losses.append(loss)
13        if epoch % 100 == 0:
14            print(f'Epoch {epoch}, Loss: {loss}')
15    return w0, w1, losses

```

Yêu cầu 4: Đánh giá hiệu suất mô hình và trực quan hóa

Sau khi mô hình đã được huấn luyện, ta cần sẽ vẽ biểu đồ Learning Curve để trực quan hóa quá trình học của mô hình. Learning Curve thể hiện sự thay đổi của hàm mất mát (MSE) theo số vòng lặp (epochs) trong quá trình huấn luyện.

```

1 import matplotlib.pyplot as plt
2 def plot_learning_curve(losses):
3     plt.figure(figsize=(10, 6))
4     plt.plot(range(len(losses)), losses, label='Training Loss (MSE)')
5     plt.xlabel('Epochs')
6     plt.ylabel('MSE Loss')
7     plt.title('Learning Curve')
8     plt.legend()
9     plt.grid()
10    plt.show()

```

Cuối cùng, ta cần đánh giá mô hình bằng cách tính toán các giá trị dự đoán trên tập kiểm tra và so sánh với giá trị thực tế. Trong bài toán hồi quy tuyến tính, các chỉ số đánh giá phổ biến bao gồm:

- Mean Absolute Error (MAE):

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Root Mean Squared Error (RMSE):

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- R-squared (R^2):

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Trong đó, $\bar{y}$ là giá trị trung bình của biến mục tiêu

Hãy hoàn thiện hàm `evaluate()` để tính toán các chỉ số đánh giá hiệu suất mô hình:

```

1 def evaluate(y_true, y_pred):
2     """
3     Params:
4         y_true: np.array, shape (n_samples, ) - true sales
5         y_pred: np.array, shape (n_samples, ) - predicted sales
6     Returns:
7         mae: float - Mean Absolute Error
8         mse: float - Mean Squared Error
9         rmse: float - Root Mean Squared Error
10        r2: float - R-squared
11    """
12    pass

```


Yêu cầu 5: Điều chỉnh siêu tham số và thử nghiệm với các đặc trưng khác

Việc lựa chọn các siêu tham số như learning rate hay số vòng lặp sẽ ảnh hưởng lớn đến hiệu suất của mô hình. Hãy thử nghiệm với các giá trị khác nhau và quan sát sự thay đổi của hàm mất mát cũng như các chỉ số đánh giá hiệu suất mô hình.

Ngoài ra, hãy thử sử dụng các đặc trưng khác trong tập dữ liệu Advertising như *Radio* hoặc *Newspaper* để xây dựng mô hình hồi quy tuyến tính. So sánh hiệu suất của các mô hình sử dụng các đặc trưng khác nhau và rút ra nhận xét.

Yêu cầu 6: Mở rộng số lượng đặc trưng đầu vào

Có thể thấy, việc sử dụng một đặc trưng đầu vào sẽ giới hạn khả năng của mô hình trong việc nắm bắt các mối quan hệ phức tạp giữa các biến. Hãy mở rộng mô hình để hỗ trợ nhiều đặc trưng đầu vào hơn, từ đó cải thiện hiệu suất dự đoán. Hãy cập nhật các hàm `predict()`, `gradient()` và `fit()` để làm việc với ma trận đặc trưng đầu vào thay vì một vector đơn.

Yêu cầu 7: Tổ chức lại code bằng Python Class

Từ các bài trên, các bạn đã triển khai mô hình hồi quy tuyến tính đơn bằng cách sử dụng các hàm riêng lẻ và thực hiện huấn luyện một cách thủ công. Tiếp theo các bạn hãy tổ chức lại code bằng cách gói gọn các chức năng vào một Python class để code dễ bảo trì, tăng khả năng tái sử dụng và phù hợp hơn trong thực tế.

Hãy hoàn thiện class `LinearRegression` dưới đây với các phương thức đã triển khai ở các bài trên:

```
1 class LinearRegression:
2     def __init__(self, learning_rate=0.01, epochs=1000, bias=True):
3         self.lr = learning_rate
4         self.epochs = epochs
5         self.weights = None
6         self.bias = bias
7
8     def predict(self, X):
9         pass
10
11    def loss_function(self, y_true, y_pred):
12        pass
13
14    def gradient(self, X, y):
15        pass
16
17    def fit(self, X, y):
18        pass
```

Bài 4: Mini-batch/Stochastic Gradient Descent

Trong các bài trước, ta đã làm quen với thuật toán Gradient Descent cơ bản (còn gọi là *Batch Gradient Descent*). Nếu bạn để ý, có thể thấy toàn bộ lượng dữ liệu trong tập huấn luyện sẽ được sử dụng để cập nhật trọng số trong mỗi lần lặp. Trên thực tế, khi lượng dữ liệu trở nên quá lớn, việc tính toán gradient trên toàn bộ dữ liệu sẽ trở nên rất tốn kém về mặt tính toán và thời gian. Để giải quyết vấn đề này, một số biến thể của Gradient Descent đã được đề xuất, bao gồm:

- **Mini-batch Gradient Descent:** Thay vì cập nhật trọng số dựa trên toàn bộ dữ liệu, thuật toán chỉ sử dụng một lượng nhỏ dữ liệu để cập nhật trọng số.

Ý tưởng chính: Chia tập dữ liệu huấn luyện thành nhiều mini-batch có kích thước m (thường là 32, 64, 128, 256,...). Với mỗi mini-batch, ta tính toán gradient và cập nhật trọng số ngay lập tức thay vì phải chờ xử lý hết toàn bộ dữ liệu.

Quy tắc cập nhật cho mini-batch thứ i :

$$\mathbf{w} := \mathbf{w} - \alpha \cdot \frac{1}{m} \sum_{j \in \text{batch}_i} \nabla_{\mathbf{w}} L(\mathbf{w}; x_j, y_j)$$

Trong đó m là kích thước của mini-batch, batch_i là tập các mẫu trong mini-batch thứ i .

Ưu điểm:

- Giảm thiểu thời gian tính toán và bộ nhớ cần thiết so với Batch GD.
- Cập nhật trọng số thường xuyên hơn, giúp mô hình hội tụ nhanh hơn.
- Có thể tận dụng tối ưu hóa phần cứng (vectorization, GPU parallel processing).
- Độ nhiễu vừa phải trong gradient giúp thoát khỏi các local minima và saddle points.

Nhược điểm:

- Gradient ước lượng có độ nhiễu hơn so với Batch GD.
- Cần điều chỉnh thêm siêu tham số `batch_size`.
- Đường đi tới nghiệm tối ưu không trơn tru như Batch GD.

- **Stochastic Gradient Descent (SGD):** Là trường hợp đặc biệt của Mini-batch Gradient Descent, trong đó kích thước mini-batch bằng 1. Mỗi lần cập nhật trọng số, thuật toán chỉ sử dụng một điểm dữ liệu ngẫu nhiên để tính toán gradient.

Ý tưởng chính: Với mỗi mẫu dữ liệu (x_i, y_i) được chọn ngẫu nhiên, ta tính gradient và cập nhật trọng số ngay lập tức:

$$\mathbf{w} := \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} L(\mathbf{w}; x_i, y_i)$$

Ưu điểm:

- Cực kỳ nhanh trong mỗi lần cập nhật (chỉ xử lý 1 mẫu).
- Bộ nhớ yêu cầu tối thiểu.

- Có thể học online (cập nhật mô hình khi dữ liệu mới đến).
- Độ nhiễu cao giúp thoát khỏi local minima dễ dàng hơn.
- Phù hợp với tập dữ liệu cực lớn (big data).

Nhược điểm:

- Gradient rất nhiễu, khiến quá trình hội tụ không ổn định.
- Có thể "dao động" mạnh xung quanh điểm tối ưu mà không hội tụ chính xác.
- Không tận dụng được vectorization, dẫn đến tốc độ tính toán chậm hơn Mini-batch GD trong thực tế.
- Loss function có thể dao động nhiều trong quá trình huấn luyện.

Lưu ý: Trong thực tế, Mini-batch Gradient Descent được sử dụng phổ biến nhất do cân bằng tốt giữa tốc độ, độ chính xác và khả năng tận dụng phần cứng. Kích thước batch thường được chọn là lũy thừa của 2 để tối ưu hóa hiệu suất tính toán trên GPU.

Hàm dưới đây sẽ giúp bạn tạo ra các mini-batch từ dữ liệu huấn luyện, để hiểu cách hoạt động của hàm này, hãy tìm hiểu từ khóa `yield` và khái niệm **generator** trong Python.

```

1 def batch_generator(X, y, batch_size=32, shuffle=True):
2     if shuffle:
3         idx = np.random.permutation(len(y))
4         X, y = X[idx], y[idx]
5     for i in range(0, len(y), batch_size):
6         yield X[i:i+batch_size], y[i:i+batch_size]
```

Yêu cầu 1: Cập nhật class `LinearRegression` để hỗ trợ Mini-batch.

- Thêm các tham số `batch_size` và `shuffle` vào hàm khởi tạo.
- Triển khai hàm `batch_generator` để tạo ra các mini-batch từ dữ liệu.
- Cập nhật hàm `fit` để sử dụng mini-batch trong quá trình huấn luyện.

Yêu cầu 2: So sánh hiệu suất giữa các phương pháp.

- Thực hiện so sánh hiệu suất giữa Mini-batch Gradient Descent và Gradient Descent thông thường trên bộ dữ liệu Advertising.
- Sau đó, đặt `batch_size = 1` để chuyển sang Stochastic Gradient Descent và so sánh hiệu suất giữa cả 3 phương pháp.